

Week 3 · Day 4 · Security Groups & NACLs

Launch a Bastion Host & Test Private Subnet Access



Time

75 min



Objective

Launch an EC2 bastion host in a public subnet, SSH into it, and from there attempt to connect to a private subnet EC2 instance — demonstrating the bastion pattern used in production.

✓ Prerequisites — Complete Before Starting

Custom VPC with route tables from Days 1-3 · SSH client (Terminal on Mac/Linux; PuTTY or Windows Terminal on Windows)



Portfolio Goal — What You Will Post

A working bastion host with successful SSH connection chain is one of the strongest portfolio pieces for a junior cloud role — document it with screenshots.

Lab Steps

Step 1 Launch the Bastion Host (Public Subnet)

1. Go to EC2 → Launch Instance.
2. Name: Bastion-Host.
3. AMI: Amazon Linux 2 (Free Tier eligible).
4. Instance type: t2.micro (Free Tier).
5. Key pair: Create a new key pair → name it: bastion-key → RSA → .pem format. DOWNLOAD AND SAVE.
6. Network settings: VPC: OluTech-Production-VPC | Subnet: Public-Subnet-AZ-A.
7. Auto-assign Public IP: Enable.
8. Security Group: Select existing → SG-WebServers (has SSH on port 22 from your IP).
9. Click Launch Instance. Wait 2 minutes for it to enter 'running' state.

Step 2 Launch a Private Instance

1. Launch a second EC2 instance: Name: Private-App-Server.
2. Same AMI and type (t2.micro).
3. Use the same key pair: bastion-key (important — same key for the hop).

4. Network: VPC: OluTech-Production-VPC | Subnet: Private-Subnet-App-AZ-A.
5. Auto-assign Public IP: DISABLE.
6. Security Group: Create new → Allow SSH from SG-WebServers only (your bastion SG).
7. Launch instance. Note: it has no public IP — you cannot reach it directly.

Step 3 SSH into the Bastion Host

1. Get the Bastion Host Public IP from the EC2 console.
2. On Mac/Linux: open Terminal. Run: `chmod 400 bastion-key.pem`
3. Run: `ssh -i bastion-key.pem ec2-user@<BASTION-PUBLIC-IP>`
4. You should see a login prompt and the Amazon Linux welcome message.
5. On Windows: Use PuTTY or Windows Terminal with your .pem file.
6. Screenshot your successful connection.

Step 4 Connect to the Private Instance via Bastion

1. From inside the bastion host terminal, note the private IP of your Private-App-Server (from EC2 console).
2. Copy your .pem key to the bastion host: on your local machine run: `scp -i bastion-key.pem bastion-key.pem ec2-user@<BASTION-IP>:/home/ec2-user/`
3. From the bastion terminal: `chmod 400 bastion-key.pem`
4. Then: `ssh -i bastion-key.pem ec2-user@<PRIVATE-APP-SERVER-PRIVATE-IP>`
5. You are now inside a private subnet instance — only reachable via the bastion!
6. Run: `hostname` and `whoami` to confirm. Screenshot.

Step 5 IMPORTANT — Clean Up to Avoid Charges

1. After completing the lab: Go to EC2 → Instances.
2. Select both instances → Instance State → Terminate.
3. Confirm termination. This ensures no charges beyond the lab session.

Screenshot Checklist

Take these screenshots and save them to your portfolio folder:

- ☐ EC2 console showing both instances running (one public IP, one private only)
- ☐ Successful SSH connection to Bastion Host (terminal prompt)
- ☐ Successful SSH hop from bastion to private instance (terminal prompt showing different private IP)
- ☐ EC2 instances terminated (to show professional cleanup habits)

 Portfolio Post — Copy & Customise

LinkedIn: 'Bastion host pattern in practice — SSHed into a public EC2, then hopped to a private subnet instance unreachable from the internet. This is how production systems are securely administered. Zero direct internet access to private resources. #AWS #EC2 #CloudSecurity #BastionHost'

 Bonus Challenge — Push Your Skills Further

Investigate AWS Systems Manager Session Manager — this is the modern replacement for bastion hosts. Read how it works and write 3 advantages over the bastion approach. Add to your portfolio as a comparison note.

 Lab Complete — Mark Off Your Progress

Lab Steps Completed	☐
All Screenshots Taken	☐
Portfolio Artifact Posted	☐
Bonus Challenge Attempted	☐